

PROTOTYPE UNIT LINKED

July 12, 2026

1 PROTOTYPE UNIT-LINKED LIABILITIES

The purpose of this notebook is to showcase how unit-linked pension liabilities work inside the OSEM model, using the same production classes and loop helpers that `main.py` calls when `liability_mode=unit_linked`.

Unit-linked products differ from the static cash-flow liability mode: they do **not** upload a pre-scheduled outflow vector. Cash flows emerge each period from policy state (account value, premiums, fees) and stochastic mortality / lapse liquidations.

Detailed methodology: [Documentation/Unit_Linked_Methodology.md](#) and [Liability_Dev/Unit_Linked_Methodology.html](#).

```
[113]: import os
import random
import numpy as np
import pandas as pd
import datetime as dt
```

```
[114]: from ConfigurationClass import Configuration
from ImportData import (
    get_configuration,
    get_settings,
    get_unit_linked_policies,
    get_unit_linked_fund,
    get_society,
)
from LiabilityClasses import UnitLinkedPolicy, UnitLinkedFund, UnitLinkedPortfolio
from SocietyClass import Society
from MainLoop import (
    capitalize_policies,
    apply_premiums,
    apply_admin_fees,
    apply_mortality,
    apply_lapse,
    process_unit_linked_period,
)
```

1.1 Input files

Unit-linked runs need the usual ALM configuration plus three liability-specific files under `Input/`.

1.1.1 Parameters.csv

Adds two fields used by the UL path:

- `liability_mode` ... `cashflow` (default) or `unit_linked`
- `random_seed` ... integer seed for reproducible mortality / lapse draws

Other parameters (modelling date, projection length, EIOPA files) are shared with the asset side.

1.1.2 Unit_Linked_Policies.csv

In-force policy file at modelling date. Columns:

- `Policy_ID` ... unique identifier
- `Birth_Date` ... for age-based mortality (`dd/mm/yyyy`)
- `Is_Female` ... 0/1 selects mortality column
- `Is_Guaranteed` ... 0/1; if 1, GV is capitalized with MV
- `Premium` ... opening annual premium
- `MV` ... opening account / reserve value
- `GV` ... opening guaranteed value

1.1.3 Unit_Linked_Fund.csv

Single-pool fund parameters (one row in the MVP):

- `Fund_ID`
- `Lapse_Rate` ... annual lapse probability
- `Admin_Fee` ... annual fee as share of MV
- `Entry_Fee` ... share of gross premium to insurer
- `Premium_Growth` ... annual premium growth rate

1.1.4 mortality.csv

Age-indexed annual mortality rates with columns `AGE`, `MALE`, `FEMALE` (already used by `Society`).

Set up the base folder

```
[115]: base_folder = os.getcwd() # Get current working directory
```

Most of the run settings are saved in the configuration file:

```
[116]: conf: Configuration
conf = get_configuration(os.path.join(base_folder, "ALM.ini"), os)
```

These lines of code extract the absolute locations of inputs:

```
[117]: parameters_file = conf.input_parameters
ul_policies_file = conf.input_unit_linked_policies
ul_fund_file = conf.input_unit_linked_fund
```

```
mortality_file = conf.input_mortality
```

```
[118]: paramfile = pd.read_csv("Input/Parameters.csv")
paramfile.index = paramfile["Parameter"]
display(paramfile)
```

	Parameter	Value
Parameter		
EIOPA_param_file	EIOPA_param_file	Input/Param_no_VA.csv
EIOPA_curves_file	EIOPA_curves_file	Input/Curves_no_VA.csv
country	country	Slovenia
run_type	run_type	Risk Neutral
n_proj_years	n_proj_years	50
Precision	Precision	1E-10
Tau	Tau	0.0001
compounding	compounding	-1
Modelling_Date	Modelling_Date	29/04/2023
liability_mode	liability_mode	unit_linked
random_seed	random_seed	42

The settings object holds modelling date, projection horizon, liability mode, and random seed.

```
[119]: settings = get_settings(parameters_file)
print("modelling_date:", settings.modelling_date)
print("liability_mode:", settings.liability_mode)
print("random_seed:", settings.random_seed)
print("end_date:", settings.end_date)
```

```
modelling_date: 2023-04-29
liability_mode: unit_linked
random_seed: 42
end_date: 2073-04-29
```

1.2 Loading unit-linked objects

A Python generator reads the policy CSV into UnitLinkedPolicy instances keyed by policy_id.

```
[120]: policy_generator = get_unit_linked_policies(ul_policies_file)
ul_policies = {policy.policy_id: policy for policy in policy_generator}
display(ul_policies)
```

```
{1001: UnitLinkedPolicy(policy_id=1001, birth_date=datetime.date(1970, 3, 15),
↳ is_female=True, is_guaranteed=False, premium=5000.0, mv=120000.0, gv=0.0),
1002: UnitLinkedPolicy(policy_id=1002, birth_date=datetime.date(1965, 7, 22),
↳ is_female=False, is_guaranteed=True, premium=8000.0, mv=250000.0, gv=200000.0),
1003: UnitLinkedPolicy(policy_id=1003, birth_date=datetime.date(1980, 1, 10),
↳ is_female=True, is_guaranteed=False, premium=3000.0, mv=75000.0, gv=0.0),
1004: UnitLinkedPolicy(policy_id=1004, birth_date=datetime.date(1955, 11, 5),
↳ is_female=False, is_guaranteed=False, premium=6000.0, mv=180000.0, gv=0.0),
```

```
1005: UnitLinkedPolicy(policy_id=1005, birth_date=datetime.date(1975, 9, 18),
↳is_female=True, is_guaranteed=False, premium=4500.0, mv=95000.0, gv=0.0)}
```

Inspect a single policy (static metadata is frozen after import):

```
[121]: example_id = sorted(ul_policies.keys())[0]
policy = ul_policies[example_id]
print(policy)
print("Age at modelling date:", policy.age_at(settings.modelling_date))
```

```
UnitLinkedPolicy(policy_id=1001, birth_date=datetime.date(1970, 3, 15),
is_female=True, is_guaranteed=False, premium=5000.0, mv=120000.0, gv=0.0)
Age at modelling date: 53
```

Fund parameters and mortality society:

```
[122]: ul_fund = get_unit_linked_fund(ul_fund_file)
display(ul_fund)

society = get_society(mortality_file)
print("Male q at age 50:", society.mortality_rate(50, is_female=False))
print("Female q at age 50:", society.mortality_rate(50, is_female=True))
```

```
UnitLinkedFund(fund_id=1, lapse_rate=0.03, admin_fee=0.005, entry_fee=0.02,
↳premium_growth=0.02)
```

```
Male q at age 50: 0.020122
Female q at age 50: 0.009009
```

1.3 Portfolio wrapper and state DataFrames

UnitLinkedPortfolio mirrors the equity/bond portfolio pattern. Simulation state lives in DataFrames (rows = policy_id, columns = modelling dates), not on the frozen dataclasses.

```
[123]: ul_ptf = UnitLinkedPortfolio(ul_policies)
ul_mv_df, ul_gv_df, ul_premium_df, ul_active_df = ul_ptf.
↳init_policy_state_to_dataframe(
    modelling_date=settings.modelling_date
)

print("Opening UL reserve:", ul_ptf.total_reserve(ul_mv_df, ul_active_df,
↳settings.modelling_date))
display(ul_mv_df)
display(ul_premium_df)
display(ul_active_df)
```

Opening UL reserve: 720000.0

```
2023-04-29
1001    120000.0
1002    250000.0
```

1003	75000.0
1004	180000.0
1005	95000.0

2023-04-29	
1001	5000.0
1002	8000.0
1003	3000.0
1004	6000.0
1005	4500.0

2023-04-29	
1001	1.0
1002	1.0
1003	1.0
1004	1.0
1005	1.0

1.4 Dual cash accounts

- `bank_account` — fund / policyholder pool cash (flows into `trade()` in the full ALM run)
- `company_account` — insurer fee revenue (entry fee and admin fee; **not** passed through trading)

In this notebook we track toy cash balances to illustrate the methodology without running the full asset ALM loop.

```
[124]: previous_date = settings.modelling_date
bank_account = pd.DataFrame(data=[0.0], columns=[previous_date])
company_account = pd.DataFrame(data=[0.0], columns=[previous_date])
display(bank_account)
display(company_account)
```

2023-04-29	
0	0.0

2023-04-29	
0	0.0

1.5 Period length

Modelling steps need not be exactly one year. OSEM uses the same convention as the asset side:

$$\Delta t = \frac{\text{currentDate} - \text{previousDate}}{365.25}$$

Annual fund rates and mortality are converted to the elapsed period (see steps below).

```
[125]: current_date = previous_date + dt.timedelta(days=365)
time_frac = (current_date - previous_date).days / 365.25
print("previous_date:", previous_date)
```

```
print("current_date:", current_date)
print("time_frac:", time_frac)
```

```
previous_date: 2023-04-29
current_date: 2024-04-28
time_frac: 0.999315537303217
```

1.6 Where UL sits in the ALM main loop

In `main.py`, after equities/bonds are marked to market and `Portfolio return` is computed, and `before proportional trade()`:

1. Capitalize MV (and GV if guaranteed) by the **period** portfolio return
2. Grow premiums; credit gross premium to `bank_account`; credit entry fee to `company_account`; add net to MV
3. Deduct admin fee from MV; credit `company_account`
4. Stochastic mortality (full MV liquidation if drawn)
5. Stochastic lapse on survivors
6. UL reserve = sum of active MV

This notebook uses an illustrative period return `r_period` (in the full run it comes from the asset portfolio).

```
[126]: # Illustrative period return from the asset MTM step in main.py
# (that value already embeds time_frac on the asset side)
r_period = 0.05
print("Illustrative portfolio_return (period factor):", r_period)
```

```
Illustrative portfolio_return (period factor): 0.05
```

1.7 Step-by-step walkthrough

Carry forward previous state into the new date column, then apply each helper used by `process_unit_linked_period`.

```
[127]: # Working copies so cells remain re-runnable independently after reload
mv_df = ul_mv_df.copy()
gv_df = ul_gv_df.copy()
premium_df = ul_premium_df.copy()
active_df = ul_active_df.copy()

mv_df[current_date] = mv_df[previous_date]
gv_df[current_date] = gv_df[previous_date]
premium_df[current_date] = premium_df[previous_date]
active_df[current_date] = active_df[previous_date]
bank_account[current_date] = bank_account[previous_date]
company_account[current_date] = company_account[previous_date]
```

1.7.1 1) Capitalize

Active policies:

$$MV_t = MV_t * (1 + r_t)$$

Where: - r_t ... is the calculated portfolio return at time t . - MV_t^i ... is the market value of the policy i at time t .

Guaranteed policies also capitalize GV the same way.

```
[128]: print("MV before capitalize:")
display(mv_df[[previous_date, current_date]])

mv_df, gv_df = capitalize_policies(
    mv_df, gv_df, active_df, ul_policies, current_date,
    portfolio_return=r_period
)

print("MV after capitalize:")
display(mv_df[[current_date]])
print("GV after capitalize (guaranteed policies only change):")
display(gv_df[[current_date]])
```

MV before capitalize:

	2023-04-29	2024-04-28
1001	120000.0	120000.0
1002	250000.0	250000.0
1003	75000.0	75000.0
1004	180000.0	180000.0
1005	95000.0	95000.0

MV after capitalize:

	2024-04-28
1001	126000.0
1002	262500.0
1003	78750.0
1004	189000.0
1005	99750.0

GV after capitalize (guaranteed policies only change):

	2024-04-28
1001	0.0
1002	210000.0
1003	0.0
1004	0.0
1005	0.0

1.7.2 2) Premiums and entry fee

$$P_t^{i,G} = P_{t-1}^{i,G} * (1 + pg_t)^{\Delta t}$$

$$F_t^{i,P} = P_t^{i,G} * f_t^P$$

$$P_t^{i,N} = P_t^{i,G} - F_t^{i,P}$$

$$b_t^c = b_t^c + F_t^{i,P}$$

$$MV_t^i = MV_t^i + P_t^{i,N}$$

Where: - pg_t ... premium growth for the period t . - Δt ... is the time fraction or the length of the period (in years) - $P_t^{i,G}$... is the gross premium for the policy i at time t - f_t^P ... is the entry fee charged by the asset manager at time t - $F_t^{i,P}$... is the size of the the entry fee, collected at time t in policy i - $P_t^{i,N}$... is the net premium for the policy i at time t - b_t^c ... is the company bank account at time t - MV_t^i ... is the market value of policy i at time t

```
[129]: mv_df, premium_df, gross_premium, entry_fee = apply_premiums(
        mv_df, premium_df, active_df, ul_fund, current_date, time_frac
    )
    bank_account[current_date] += gross_premium
    company_account[current_date] += entry_fee

    print("gross_premium:", gross_premium)
    print("entry_fee (to company_account):", entry_fee)
    print("bank_account:", float(bank_account.loc[0, current_date]))
    print("company_account:", float(company_account.loc[0, current_date]))
    display(mv_df[[current_date]])
    display(premium_df[[current_date]])
```

```
gross_premium: 27029.633633546677
entry_fee (to company_account): 540.5926726709337
bank_account: 27029.633633546677
company_account: 540.5926726709337
```

```
2024-04-28
1001 130997.932257
1002 270496.691611
1003 81748.759354
1004 194997.518708
1005 104248.139031
```

```
2024-04-28
1001 5099.930874
1002 8159.889399
```


1003 3059.958525
 1004 6119.917049
 1005 4589.937787

1.7.3 3) Admin fee

Actual administration fee:

$$adminFee_t = 1 - (1 - f_t^A)^{\Delta t}$$

$$MV_t^i = MV_t^i * (1 - adminFee_t)$$

$$F_t^{i,A} = MV_t^i * adminFee_t$$

$$b_t^c = b_t^c + F_t^{i,A}$$

Where: - $f^A \dots$ is the relative size of the administration fee (on a yearly basis) - $F_t^{i,A} \dots$ is the absolute size of the administration fee collected by the company - $adminFee_t \dots$ is the relative size of the administration fee given the length of the period t

```
[130]: mv_df, admin_fee = apply_admin_fees(mv_df, active_df, ul_fund, current_date,
      ↪time_frac)
      company_account[current_date] += admin_fee

      print("admin_fee (to company_account):", admin_fee)
      print("company_account:", float(company_account.loc[0, current_date]))
      display(mv_df[[current_date]])
```

```
admin_fee (to company_account): 3909.7739834191093
company_account: 4450.366656090043
```

```
2024-04-28
1001 130343.389790
1002 269145.131561
1003 81340.294627
1004 194023.196787
1005 103727.254213
```

1.7.4 4) Mortality sampling (stochastic)

For each **active** policy (sorted by `policy_id`):

$$mF_t^i = 1 - (1 - q_t^i)^{\Delta t}$$

$u \sim \text{Uniform}(0, 1)$

If the fund is guaranteed then:

$$b_t^f = b_t^f - \max(MV_t^i, GV_t^i) I_{u \leq mF_t^i}$$

Otherwise:

$$b_t^f = b_t^f - MV_t^i I_{u \leq mF_t^i}$$

$$MV_t^i = MV_t^i I_{u > mF_t^i}$$

$$GV_t^i = GV_t^i I_{u > mF_t^i}$$

Where: - b_t^f ... is the fund's own bank account - mF_t^i ... is the the probability, the client in policy i dies within period t . - q_t^i ... is the probability that a person that is the same age as client in policy i in the same year as the start of period t .

Using a forced RNG that always survives makes the path deterministic for teaching.

```
[131]: class AlwaysSurvive:
        def random(self):
            return 0.99

mv_df, active_df, death_total, deaths = apply_mortality(
    mv_df, active_df, ul_policies, society, current_date, time_frac,
    ↪ AlwaysSurvive()
)
bank_account[current_date] -= death_total

print("deaths:", deaths, "death_total:", death_total)
display(active_df[[current_date]])
```

deaths: 0 death_total: 0.0

	2024-04-28
1001	1.0
1002	1.0
1003	1.0
1004	1.0
1005	1.0

1.7.5 5) Lapse sampling (survivors only)

Same Bernoulli pattern with period-scaled `lapse_rate` on policies that remain active after mortality.

For each **active** policy (sorted by `policy_id`):

$$lF_t^i = 1 - (1 - l_t)^{\Delta t}$$

$u \sim \text{Uniform}(0, 1)$

If the fund is guaranteed then:

$$b_t^f = b_t^f - \max(MV_t^i, GV_t^i) I_{u \leq lF_t^i}$$

Otherwise:

$$b_t^f = b_t^f - MV_t^i I_{u \leq lF_t^i}$$

$$MV_t^i = MV_t^i I_{u > lF_t^i}$$

$$GV_t^i = GV_t^i I_{u > lF_t^i}$$

Where: - b_t^f ... is the fund's own bank account - lF_t^i ... is the the probability, the client in policy i lapses within period t . - l_t ... is the probability that a person lapses in the same year as the start of period t .

```
[132]: mv_df, active_df, surrender_total, lapses = apply_lapse(
        mv_df, active_df, ul_fund, current_date, time_frac, AlwaysSurvive()
    )
    bank_account[current_date] -= surrender_total

    print("lapses:", lapses, "surrender_total:", surrender_total)
    print("UL reserve:", ul_ptf.total_reserve(mv_df, active_df, current_date))
    print("bank_account:", float(bank_account.loc[0, current_date]))
    print("company_account:", float(company_account.loc[0, current_date]))
```

```
lapses: 0 surrender_total: 0.0
UL reserve: 778579.2669774566
bank_account: 27029.633633546677
company_account: 4450.366656090043
```

1.8 One-shot orchestrator (process_unit_linked_period)

The production helper carry-forwards state, runs steps 1–5, and returns cash-flow totals. Signs for summary reporting are applied in `main.py` (benefits logged negative).

```
[133]: mv2, gv2, prem2, act2 = ul_ptf.init_policy_state_to_dataframe(previous_date)

mv2, gv2, prem2, act2, cfs = process_unit_linked_period(
    current_date=current_date,
    previous_date=previous_date,
    portfolio_return=r_period,
```

```

        time=time_frac,
        mv_df=mv2,
        gv_df=gv2,
        premium_df=prem2,
        active_df=act2,
        policies=ul_policies,
        fund=ul_fund,
        society=society,
        random_seed=settings.random_seed,
        proj_period=0,
    )

display(pd.Series(cfs, name="cashflows"))
print("UL reserve:", ul_ptf.total_reserve(mv2, act2, current_date))
display(act2[[current_date]])

```

```

gross_premium      27029.633634
entry_fee           540.592673
admin_fee           3909.773983
death               269145.131561
surrender            0.000000
deaths              1.000000
lapses              0.000000
in_force            4.000000
Name: cashflows, dtype: float64

```

```
UL reserve: 509434.13541679934
```

```

2024-04-28
1001      1.0
1002      0.0
1003      1.0
1004      1.0
1005      1.0

```

1.9 Reproducibility

The same `random_seed` and `proj_period` produce the same draws. Different seeds can change liquidations.

```

[134]: def run_seeded(seed: int):
        mv, gv, prem, act = ul_ptf.init_policy_state_to_dataframe(previous_date)
        *_ , cfs = process_unit_linked_period(
            current_date=current_date,
            previous_date=previous_date,
            portfolio_return=r_period,
            time=time_frac,
            mv_df=mv,
            gv_df=gv,

```

```

        premium_df=prem,
        active_df=act,
        policies=ul_policies,
        fund=ul_fund,
        society=society,
        random_seed=seed,
        proj_period=1,
    )
    return cfs["deaths"], cfs["lapses"], cfs["in_force"]

print("seed 42:", run_seeded(42))
print("seed 42 again:", run_seeded(42))
print("seed 99:", run_seeded(99))

```

```

seed 42: (0.0, 1.0, 4.0)
seed 42 again: (0.0, 1.0, 4.0)
seed 99: (0.0, 0.0, 5.0)

```

1.10 Multi-period illustration

Roll several periods with a fixed illustrative return and the configuration seed. Track reserve, company account, and policies in force.

```

[135]: n_periods = 10
mv, gv, prem, act = ul_ptf.init_policy_state_to_dataframe(settings.
    ↪modelling_date)
company = pd.DataFrame(data=[0.0], columns=[settings.modelling_date])
bank = pd.DataFrame(data=[0.0], columns=[settings.modelling_date])

history = []
prev = settings.modelling_date
for k in range(1, n_periods + 1):
    curr = prev + dt.timedelta(days=365)
    tf = (curr - prev).days / 365.25
    company[curr] = company[prev]
    bank[curr] = bank[prev]

    mv, gv, prem, act, cfs = process_unit_linked_period(
        current_date=curr,
        previous_date=prev,
        portfolio_return=0.04,
        time=tf,
        mv_df=mv,
        gv_df=gv,
        premium_df=prem,
        active_df=act,
        policies=ul_policies,
        fund=ul_fund,
    )

```

```

        society=society,
        random_seed=settings.random_seed,
        proj_period=k,
    )
    bank[curr] += cfs["gross_premium"] - cfs["death"] - cfs["surrender"]
    company[curr] += cfs["entry_fee"] + cfs["admin_fee"]

    history.append({
        "date": curr,
        "UL reserve": ul_ptf.total_reserve(mv, act, curr),
        "Company account": float(company.loc[0, curr]),
        "Bank account": float(bank.loc[0, curr]),
        "In force": cfs["in_force"],
        "Deaths": cfs["deaths"],
        "Lapses": cfs["lapses"],
        "Gross premium": cfs["gross_premium"],
    })
    prev = curr

history_df = pd.DataFrame(history).set_index("date")
display(history_df)

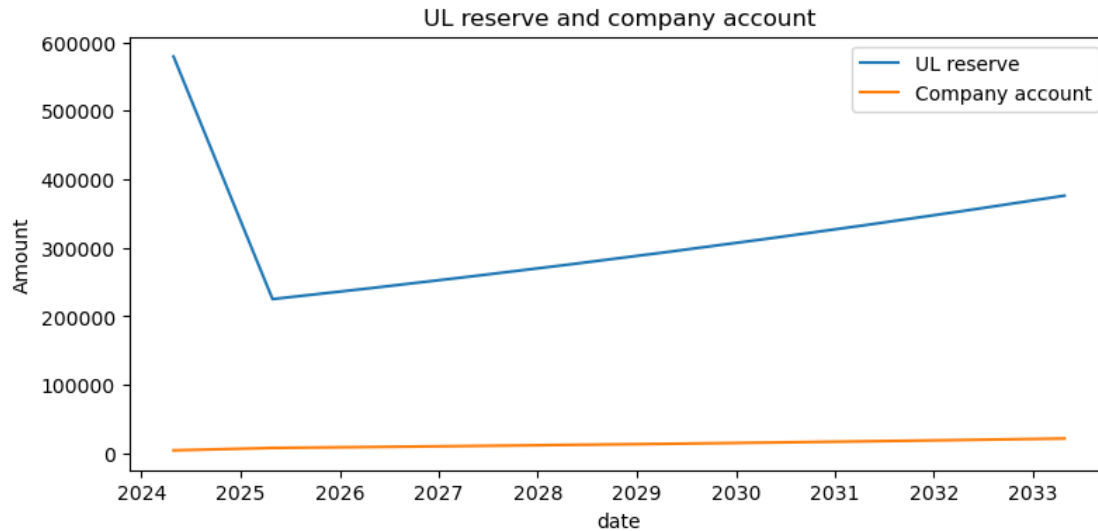
try:
    ax = history_df[["UL reserve", "Company account"]].plot(
        figsize=(9, 4), title="UL reserve and company account"
    )
    ax.set_ylabel("Amount")
except ImportError:
    print("Install matplotlib to plot this table (optional).")

```

	UL reserve	Company account	Bank account	In force	Deaths	\
date						
2024-04-28	579183.051756	4414.391235	-165202.557009	4.0	0.0	
2025-04-28	225159.003855	7955.073264	-538853.244951	2.0	0.0	
2026-04-28	241273.299730	9336.456339	-530363.926155	2.0	0.0	
2027-04-28	258113.875131	10805.800733	-521704.938349	2.0	0.0	
2028-04-27	275709.313774	12366.964947	-512872.890499	2.0	0.0	
2029-04-27	294089.260274	14023.957701	-503864.323796	2.0	0.0	
2030-04-27	313284.458393	15780.943485	-494675.710303	2.0	0.0	
2031-04-27	333326.790639	17642.248303	-485303.451575	2.0	0.0	
2032-04-26	354249.319273	19612.365624	-475743.877245	2.0	0.0	
2033-04-26	376086.328772	21695.962550	-465993.243591	2.0	0.0	

	Lapses	Gross premium
date		
2024-04-28	1.0	27029.633634
2025-04-28	2.0	21327.621836
2026-04-28	0.0	8489.318796

2027-04-28	0.0	8658.987806
2028-04-27	0.0	8832.047850
2029-04-27	0.0	9008.566703
2030-04-27	0.0	9188.613492
2031-04-27	0.0	9372.258728
2032-04-26	0.0	9559.574330
2033-04-26	0.0	9750.633654



1.11 Connection to the full ALM run

In `main.py` with `liability_mode=unit_linked`:

1. Assets still evolve first (dividends, coupons, growth, bond reprice).
2. `Portfolio return` from the asset book capitalizes UL accounts.
3. UL cash flows update `bank_account` and `company_account`.
4. Proportional `trade()` invests / disinvests using **fund** cash only (`company_account` is excluded).

Switch the mode in `Input/Parameters.csv`:

```
liability_mode,unit_linked
random_seed,42
```

Then run `python main.py` and inspect `Output/Results.csv` for the UL summary columns.

Run this notebook from the repository root so `ALM.ini` and `Input/` resolve correctly.

1.12 Out of scope for this MVP

New business, lifecycle fund migration, full DCF reserving, company-account payouts, morbidity / retirement, and multi-fund asset buckets are not modelled yet.